Lab Five-Six – Data Wrangling in R

Tips

- Complete the tasks below. Make sure to start your solutions in on a new line that starts with "**Solution**:".
- Make sure to use the Quarto Cheatsheet. This will make completing and writing up the lab *much* easier.
- Make sure to use "enter" to make your code readable, so it doesn't run off the page. I switched the Quarto document to automatically render to .pdf, so you can see what you're submitting by default. You can switch back for the highlighting by moving the html block above the pdf block in the YAML header.
- Use the pipe operator (`|>`). It makes the data wrangling tasks much easier to write.
- Use `head(...)` or `glimpse(...)` often to ensure your code did what you wanted it to and that you haven't accidentally deleted their entire dataset with a bad filter.
- Don't forget that you can copy-and-paste code when you're doing something similar as we did in class!

Timeline

- Week of 2/16: Aim to complete the data wrangling sections
- Week of 2/23: Aim to complete the data summaries sections

1 Biomedical Science Data

Kitsberg et al. (2025) collect data on the Human Cytomegalovirus (HCMV). HCMV is a member of the herpesvirus family – the same family that includes: chickenpox, mononucleosis (mono), and cold sores. Prevalence is high with a worldwide seroprevalence of 24% to 100% (Gupta and Shorman 2025). The virus persists in a dormant state for the entire life of those infected. Primary cytomegalovirus infection is often asymptomatic but can present with flu-like symptoms. The infection can reactivate due to stress, aging, or a weakened immune system.

The researchers obtained blood samples from healthy donors, males and females, aged 25-45. They infected the cells of primary (human donor's blood) or THP1 (purchased cell lines) monocytes ($n = 87$ and $n = 112$, respectively) and macrophages ($n = 93$ and $n = 170$, respectively) and counted the number of viral genomes in the nuclei.

Note that the original donor of the THP-1 cell line was a 1-year-old Japanese boy. He was treated for Acute Monocytic Leukemia at Tohoku Hospital Pediatrics (THP-1; the first cell line there). Because they are malignant cancer cells, they can divide indefinitely, breaking the usual cell managment that usually reigns in rogue cells. Therefore, these cells can divide indefinitely in a lab environment. THP-1 cells, and those like them, serve as such a reliable, uniform model for medical research and improve reproducability.

Preliminary Question: Are THP-1 cells a good stand-in for human cells?

Solution: Though they are not perfectly comparable to human cells, THP-1 cells are likely a good stand-in for research purposes. Though my knowledge of biology is limited, it is safe to assume that cancer cells behave differently from healthy human cells beyond just indefinite division. Furthermore, the origin of these cells lie in one unique patient, therefore they capture limited variation that occurs across the cells of different humans. However, despite these concerns, for reproducible research purposes, these cells are a good proxy for normal human cells. Their indefinite growth allows for countless experiments/studies to be run at a much much cheaper cost compared with normal cells, and the aforementioned concerns can likely be accounted for or addressed in any published research.

Research Question: Is the virus more prevalent in the nucleus of monocytes or macrophages, explaining why HCMV stays dormant in monocytes but goes active in macrophages?

1.1 Data Wrangling

1.1.1 Part A

Go to the website and download the source data https://www.nature.com/articles/s41467-025-68063-y#Sec30.

Describe the process – do not leave out any steps – for how you can load the data for Figures 3b and 3d from the manuscript into `R`.

Hint: You can use packages like `readxl` (Wickham and Bryan 2025) but you will find it substantially easier to open the `.xls` file using your computer's spreadsheet software and create a new `.csv` file with the data you need.

Solution: To accomplish this task, I opened the link provided and downloaded the source data. I then opened the data using Microsoft Excel and navigated to the tab entitled `Fig 3b`. I then appended the duplicates of each column so that there is only one column entitled "sample" and one column entitled "viruses within nucleus" I then clicked `file`, `save as`, renamed it to `"Lab5Fig3B"`, selected `.csv` as the format, and saved it to the my folder for Labs 5 and 6.

After doing that, I load it in using the `tidyverse` (Wickham et al. 2019) function, `read_csv()` and the full file path, skipping the blank rows.

```
dat.kitsberg = read_csv(
  "/Users/alexvogt/Documents/GitHub/lab-5-6-data-wrangling-alvogt1020/Lab5Fig3B.csv",
  skip = 1
)
```

1.1.2 Part B

The data need to be cleaned. Note that the sample type (Healthy Donor or THP-1), cell type (Monocytes or Macrophages), and treatment (12 Hours Post-Infection or Uninfected) are all recorded in one column `sample`, separated with underscores. Use `separate(...)` from the `tidyverse` package to separate the `sample` column into the `SampleType`, `CellType`, `Treatment` columns. Save the object as `dat.kitsberg.cleaned`.

Hint: Use `?separate(...)` to see how it works.

Solution: I fill in the `separate(...)` function with the appropriate arguments to break up `sample` into multiple based on the criteria given.

```
dat.kitsberg.cleaned = dat.kitsberg |> separate(
  col = "sample",
  into = c("SampleType","CellType","Treatment"),
  sep = "_"
)

dat.kitsberg.cleaned = dat.kitsberg.cleaned[, 1:4] # eliminate NA columns
```

1.1.3 Part C

The researchers are only interested in observations in the treatment group (12hpi). Remove uninfected samples from the data. Update `dat.kitsberg.cleaned` with the result.

Hint: I would use the `filter(...)` function.

Solution: I use the `filter(...)` function to select only observations for which `Treatment` takes on the value "12hpi".

```
dat.kitsberg.cleaned = dplyr::filter(
  dat.kitsberg.cleaned,
  Treatment == "12hpi"
)
```

1.1.4 Part D

When `CellType` is "MDM", it is a Monocyte-Derived Macrophages. That is, the cell type should be macrophages which is coded as "mac". Update `dat.kitsberg.cleaned` with the result.

Solution: I feed dat.kitsberg.cleaned into the `mutate(...)` function, which uses a conditional statement to recode MDM in `CellType` to mac.

```
dat.kitsberg.cleaned = dat.kitsberg.cleaned |>
  mutate(
    CellType = if_else(CellType == "MDM","mac",CellType)
  )
```

Hint: I would use the `mutate(...)` function. You will likely find `if_else(...)` or `case_when(...)` helpful.

1.2 Part E

The researchers are only interested in monocytes and macrophages. Remove any other cell types. Update `dat.kitsberg.cleaned` with the result.

Hint: I would use the `filter(...)` function.

Solution: I use `View(...)` to find the appropriate observation names, and the filter function from `dplyr` to rename the statements based on a logical *or* condition.

```
dat.kitsberg.cleaned = dplyr::filter(
                                dat.kitsberg.cleaned,
                                CellType %in% c("mac","mono")
                                )
```

1.3 Part F

Clean up the data entry. Replace the shorthand the researchers used with the neat sample type (Healthy Donor or THP-1), cell type (Monocytes or Macrophages), and treatment (12 Hours Post-Infection or Uninfected). Update `dat.kitsberg.cleaned` with the result.

Hint: I would use the `mutate(...)` function. You will likely find `if_else(...)` or `case_when(...)` helpful.

Solution:

I use `View(...)` to see what each observation is named so that I can use a series of `if_else` statements to rename the observations.

Then, I use nested `if_else(...)` statements because `SampleType` and `CellType` have two values that need to be renamed. Take `SampleType` for example: the first `if_else(...)` checks whether the value is `"primary"` and renames it `"Healthy Donor"`. The second `if_else(...)` is needed because values that are not `"primary"` still need to be checked to see whether they are `"THP1"`, and if so renames them to `"THP-1"`. Any values that fail to match the conditionals remain unchanged, which includes any missing/NA values.

```
dat.kitsberg.cleaned = dat.kitsberg.cleaned |>
                      mutate(
                      SampleType = if_else(SampleType == "primary", "Healthy Donor",
                                   if_else(SampleType == "THP1", "THP-1", SampleType)),
                      CellType = if_else(CellType == "mono", "Monocytes",
                                 if_else(CellType == "mac", "Macrophages", CellType)),
                      Treatment = if_else(Treatment == "12hpi", "12 Hours Post-Infection",
                                  if_else(Treatment == "uninfected", "Uninfected",Treatment))
                      )
```

1.4 Part G

In an `R` code block, combine your answers into one code block that completes the full cleaning process. Save the object as `dat.kitsberg.cleaned`.

Note: You will use this in the next section

Solution:

```
dat.kitsberg.cleaned = read_csv(
  "/Users/alexvogt/Documents/GitHub/lab-5-6-data-wrangling-alvogt1020/Lab5Fig3B.csv",
  skip = 1
) |>

separate(
  col = "sample",
  into = c("SampleType","CellType","Treatment"),
  sep = "_"
) |>

select(1:4) |>

dplyr::filter(Treatment == "12hpi") |>

mutate(
    CellType = if_else(CellType == "MDM","mac",CellType)
) |>

dplyr::filter(CellType == "mac" | CellType == "mono") |>

mutate(
  SampleType = if_else(SampleType == "primary", "Healthy Donor",
                                    if_else(SampleType == "THP1","THP-1",SampleType)),
                       CellType   = if_else(CellType == "mono", "Monocytes",
                                    if_else(CellType == "mac", "Macrophages", CellType)),
                       Treatment  = if_else(Treatment == "12hpi", "12 Hours Post-Infection",
                                    if_else(Treatment == "uninfected", "Uninfected",Treatment))
)
```

1.5 Data Summaries

1.5.1 Part A

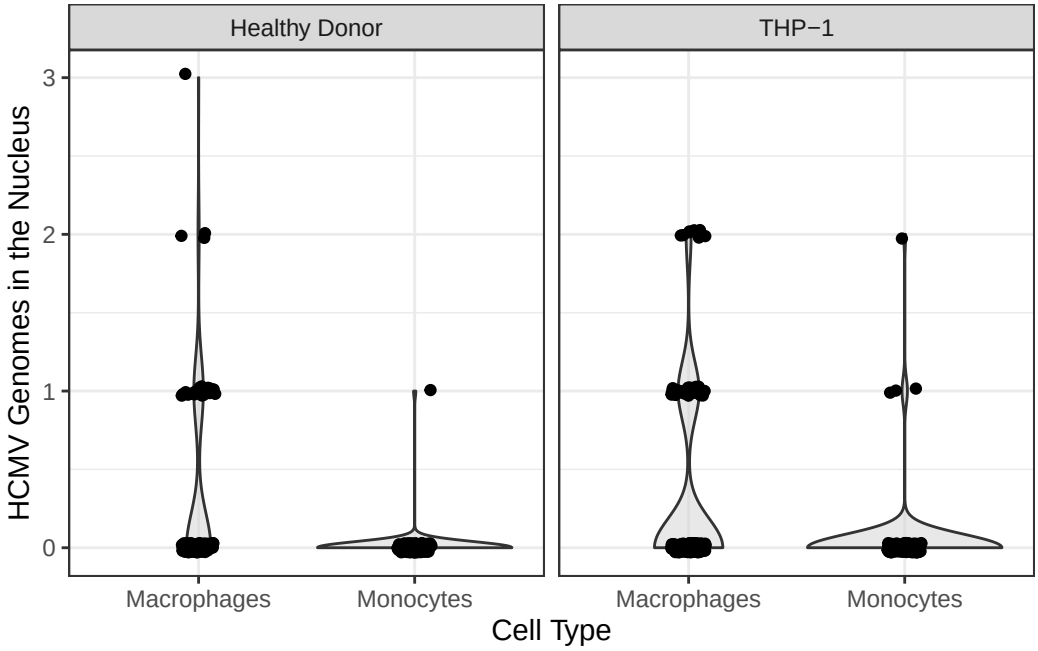
Recreate Figure 3b as depicted in the paper (see https://www.nature.com/articles/s41467-025-68063-y#Fig3). You can ignore the significance comparison because we haven't done that yet.

Hint: I would use `geom_violin(...)` for Figure 3b I also expect your plots to look nicer – better labels, better scaling, etc. Don't forget to use the proper formatting using the syntax in the Quarto cheatsheet.

Solution:

```
ggplot(data = dat.kitsberg.cleaned |> drop_na(SampleType)) + # data to plot
    geom_violin(aes(x = CellType,
                    y = `viruses within nucleus`),
                fill = "lightgrey", alpha=0.5) +
    geom_jitter(aes(x = CellType,
                    y = `viruses within nucleus`),
              width = 0.08,
              height = 0.03) +
    theme_bw() +
    labs(x = "Cell Type",
         y = "HCMV Genomes in the Nucleus")+
    facet_wrap(~SampleType)
```

Figure 1: HCMV genomes in the nucleus for monocytes and macrophages by sample type (Healthy Donor vs THP-1).



1.5.2 Part B

Compute a table containing relevant numerical summaries for Figure 3b.

Hint: I would use `group_by(...)` and `summarize(...)` to complete this. Don't forget to use the proper formatting using `knitr::kable(...)` as described in the Quarto Cheatsheet.

Solution:

```
kitsberg.summary = dat.kitsberg.cleaned |>
  group_by(SampleType, CellType) |>
  summarize(
    n = n(),
    Missing = sum(is.na(`viruses within nucleus`)),
    Mean = mean(`viruses within nucleus`, na.rm = T),
    SD = sd(`viruses within nucleus`, na.rm = T),
    Median = median(`viruses within nucleus`, na.rm = T),
    IQR = IQR(`viruses within nucleus`, na.rm = T)
  )
```

To print the table nicely, we use the `knitr::kable(...)`, as suggested.

Table 1: HCMV Genomes in the Nucleus

SampleType	CellType	n	Missing	Mean	SD	Median	IQR
Healthy Donor	Macrophages	93	0	0.37	0.60	0	1
Healthy Donor	Monocytes	87	0	0.01	0.11	0	0
THP-1	Macrophages	170	0	0.33	0.57	0	1
THP-1	Monocytes	112	0	0.04	0.25	0	0

From the numerical summary, we see that the average amount of HCMV genomes in the nucleus is significantly higher for macrophages than monocytes. Furthermore, macrophages exhibit much more spread with a standard deviation 3x that of monocytes. The violin plots further enrich our understanding of the results. We see that healthy

donors in general exhibit more spread than THP-1 donors (which aligns with the intuitive explanation that humans are more varied than artificial cells).

Note: Parts C and D removed, to align with the revised version of the lab assignment.

2 Ship Valuation

Figure 2: This is a ship-shipping ship shipping shipping ships.

Bal et al. (2025) aim to build a model for predicting the value of Supramax and Ultramax ships using both current and past market data. Their goal was to build a model that only uses information anyone can easily look up.

The researchers collected 17 years of real ship sale prices (2005–2022), covering $n = 1819$ Supramax and Ultramax ship sales.

Research Question: Can we predict Supramax and Ultramax ship prices?

2.1 Data Wrangling

2.1.1 Part A

I downloaded the data from linked in the article (https://journals.plos.org/plosone/article?id=10.1371/journal.pone.0319073). I have included the data in this lab folder as `ShipValuation-Bal25.csv`. Load the data into `R`.

Solution:

```
dat.bal = read_csv("ShipValuation-Bal25.csv")
```

2.1.2 Part B

The price of each Supramax and Ultramax ship is reported on the $\log_{10}$ scale. Create a new column called `price` that contains the price in millions of dollars.

$$\text{Price} = \frac{10^{\text{SPLog}}}{1000000}$$

Save the object as `dat.bal.cleaned`.

Hint: I would use the `mutate(...)` function.

Solution:

```r
dat.bal.cleaned = dat.bal |>
        mutate(
          price = 10 ^ SPLog / 1000000
        )
```

2.1.3 Part C

The authors recorded the country that each Supramax or Ultramax ship was built in in the columns `Japan`, `China, People's Republic Of`, `Philippines`, `Korea, South`, `Others Country Build`. These columns are marked `1` to indicate that is the country of origin and `0` otherwise. Create a new column called `OriginCountry` that contains a clean labels for the country in which the ship was built. Update `dat.bal.cleaned` with the result.

Hint: I would use the `mutate(...)` function. You will likely find `if_else(...)` or `case_when(...)` helpful.

Solution: Using `case_when(...)` in this instance is much easier given the amount of columns needed to scan as well as the character lengths.

```r
dat.bal.cleaned = dat.bal.cleaned |>
  mutate(
    OriginCountry = case_when(
      Japan == 1 ~ "Japan",
      `China, People's Republic Of` == 1 ~ "China",
      Philippines == 1 ~ "Philippines",
      `Korea, South` == 1 ~ "South Korea",
      `Others Country Build` == 1 ~ "Other",
      TRUE ~ NA
    )
  )
```

2.1.4 Part D

The authors recorded each Supramax and Ultramax ship's main engine manufacturer in the columns `Caterpillar`, `MAN-B&W`, `Wartsila`, `Mitsubishi`. These columns are marked `1` to indicate that is the manufacturer and `0` otherwise. Create a new column called `Manufacturer` that contains a clean labels for the manufacturer. Update `dat.bal.cleaned` with the result.

Hint: Follow the logic of the previous question.

Solution:

Copy and pasting the code from part C and adjusting it is the quickest approach for this.

```r
dat.bal.cleaned = dat.bal.cleaned |>
  mutate(
    Manufacturer = case_when(
      Caterpillar == 1 ~ "Caterpillar",
      `MAN-B&W` == 1 ~ "MAN-B&W",
      Wartsila == 1 ~ "Wartsila",
      Mitsubishi == 1 ~ "Mitsubishi",
      TRUE ~ NA
    )
  )
```

2.1.5 Part E

The researchers recorded whether each Supramax and Ultramax ship is currently covered by a Protection and Indemnity (P&I) Club. This can be an indicator that a ship meets the minimum seaworthiness and safety requirements of the insurer.

The authors recorded P&I clubs in the columns `Japan P&I Club`, `Gard`, `West of England`, `North of England`, `UK P&I`, `Britannia P&I`, `London P&I Club`, `Skuld`, `Steamship Mutual`, `Others`, `Unknown`. These columns are marked

`1` to indicate that the ship is insured by that P&I club and `0` otherwise. Create a new column called `PI.club` that contains a clean labels for the P&I club. Update `dat.bal.cleaned` with the result.

Hint: Follow the logic of the previous questions.

Solution:

Again, copy and pasting the code from part C and adjusting it is the quickest approach for this.

```
dat.bal.cleaned = dat.bal.cleaned |>
  mutate(
    PI.Club = case_when(
      `Japan P&I Club` == 1 ~ "Japan P&I Club",
      `Gard` == 1 ~ "Gard",
      `West of England` == 1 ~ "West of England",
      `North of England` == 1 ~ "North of England",
      `UK P&I` == 1 ~ "UK P&I",
      `Britannia P&I` == 1 ~ "Britannia P&I",
      `London P&I Club`== 1 ~ "London P&I Club",
      `Skuld` == 1 ~ "Skuld",
      `Steamship Mutual` == 1 ~ "Steamship Mutual",
      `Others` == 1 ~ "Others",
      `Unknown` == 1 ~ "Unknown",
      TRUE ~ NA
    )
  )
```

2.1.6 Part F

The researchers recorded whether each Supramax and Ultramax ship is classed by a society that requires a ship to be safe, structurally sound, and built/maintained to specific engineering standards.

The authors recorded class society in the columns `Nippon Kaiji Kyokai (IACS)`, `Lloyd's Register`, `Bureau Veritas (IACS)`, `Det Norske Veritas (IACS)`, `American Bureau of Shipping (IACS)`, `Others Class`, `With Two Classes`, `Disclassed`. These columns are marked `1` to indicate that the ship is classed by that soceity and `0` otherwise. Create a new column called `Class` that contains a clean labels for the class. Update `dat.bal.cleaned` with the result.

Hint: Follow the logic of the previous questions.

Solution:

Again, copy and pasting the code from part C and adjusting it is the quickest approach for this.

```
dat.bal.cleaned = dat.bal.cleaned |>
  mutate(
    Class = case_when(
      `Nippon Kaiji Kyokai (IACS)` == 1 ~ "Nippon Kaiji Kyokai (IACS)",
      `Lloyd's Register` == 1 ~ "Lloyd's Register",
      `Bureau Veritas (IACS)` == 1 ~ "Bureau Veritas (IACS)",
      `Det Norske Veritas (IACS)` == 1 ~ "Det Norske Veritas (IACS)",
      `American Bureau of Shipping (IACS)` == 1 ~ "American Bureau of Shipping (IACS)",
      `Others Class` == 1 ~ "Others Class",
      `With Two Classes`== 1 ~ "With Two Classes",
      `Disclassed` == 1 ~ "Disclassed",
      TRUE ~ NA
    )
  )
```

2.1.7 Part G

The researchers recorded the flag state of each Supramax and Ultramax ship. Each flag state enforces its own set of safety, environmental, and crew welfare rules and regulations. Ships under certain flag states may be more or less valuable depending on how much the flag affects the buyer's trust in the condition of the boat.

The authors recorded flag states in the columns `Panama`, `Marshall Islands`, `Liberia`, `Malta`, `Hong Kong`, `China`, `Singapore`, `Others Flag`, `With two flags`. These columns are marked `1` to indicate that the ship has that flag state and `0` otherwise. Create a new column called `FlagState` that contains a clean labels for the flag state. Update `dat.bal.cleaned` with the result.

Hint: Follow the logic of the previous questions.

Solution:

Again, copy and pasting the code from part C and adjusting it is the quickest approach for this.

```r
dat.bal.cleaned = dat.bal.cleaned |>
  mutate(
    FlagState = case_when(
      `Panama` == 1 ~ "Panama",
      `Marshall Islands` == 1 ~ "Marshall Islands",
      `Liberia` == 1 ~ "Liberia",
      `Malta` == 1 ~ "Malta",
      `Hong Kong, China` == 1 ~ "Hong Kong",
      `Singapore` == 1 ~ "Singapore",
      `Others Flag` == 1 ~ "Others Flag",
      `With two flags`== 1 ~ "With two flags",
      TRUE ~ NA
    )
  )
```

2.2 Part H

In an `R` code block, combine your answers into one code block that completes the full cleaning process. Save the object as `dat.bal.cleaned`.

Note: You will use this in the next section

```r
# Loading the data (part A)
dat.bal = read_csv("ShipValuation-Bal25.csv")

# Creating price col (part B)
dat.bal.cleaned = dat.bal |>
        mutate(
          price = 10 ^ SPLog / 1000000
        )

# Creating OriginCountry (part C)
dat.bal.cleaned = dat.bal.cleaned |>
  mutate(
    OriginCountry = case_when(
      Japan == 1 ~ "Japan",
      `China, People's Republic Of` == 1 ~ "China",
      Philippines == 1 ~ "Philippines",
      `Korea, South` == 1 ~ "South Korea",
      `Others Country Build` == 1 ~ "Other",
      TRUE ~ NA
    )
  )

```

```r
# Creating Manufacturer (part D)
dat.bal.cleaned = dat.bal.cleaned |>
  mutate(
    Manufacturer = case_when(
      Caterpillar == 1 ~ "Caterpillar",
      `MAN-B&W` == 1 ~ "MAN-B&W",
      Wartsila == 1 ~ "Wartsila",
      Mitsubishi == 1 ~ "Mitsubishi",
      TRUE ~ NA
    )
  )

# Creating PI.club (part E)
dat.bal.cleaned = dat.bal.cleaned |>
  mutate(
    PI.Club = case_when(
      `Japan P&I Club` == 1 ~ "Japan P&I Club",
      `Gard` == 1 ~ "Gard",
      `West of England` == 1 ~ "West of England",
      `North of England` == 1 ~ "North of England",
      `UK P&I` == 1 ~ "UK P&I",
      `Britannia P&I` == 1 ~ "Britannia P&I",
      `London P&I Club`== 1 ~ "London P&I Club",
      `Skuld` == 1 ~ "Skuld",
      `Steamship Mutual` == 1 ~ "Steamship Mutual",
      `Others` == 1 ~ "Others",
      `Unknown` == 1 ~ "Unknown",
      TRUE ~ NA
    )
  )

# Creating Class (part F)
dat.bal.cleaned = dat.bal.cleaned |>
  mutate(
    Class = case_when(
      `Nippon Kaiji Kyokai (IACS)` == 1 ~ "Nippon Kaiji Kyokai (IACS)",
      `Lloyd's Register` == 1 ~ "Lloyd's Register",
      `Bureau Veritas (IACS)` == 1 ~ "Bureau Veritas (IACS)",
      `Det Norske Veritas (IACS)` == 1 ~ "Det Norske Veritas (IACS)",
      `American Bureau of Shipping (IACS)` == 1 ~ "American Bureau of Shipping (IACS)",
      `Others Class` == 1 ~ "Others Class",
      `With Two Classes`== 1 ~ "With Two Classes",
      `Disclassed` == 1 ~ "Disclassed",
      TRUE ~ NA
    )
  )

# Creating FlagState (part G)
dat.bal.cleaned = dat.bal.cleaned |>
  mutate(
    FlagState = case_when(
      `Panama` == 1 ~ "Panama",
      `Marshall Islands` == 1 ~ "Marshall Islands",
      `Liberia` == 1 ~ "Liberia",
      `Malta` == 1 ~ "Malta",
      `Hong Kong, China` == 1 ~ "Hong Kong",
```

```
      `Singapore` == 1 ~ "Singapore",
      `Others Flag` == 1 ~ "Others Flag",
      `With two flags`== 1 ~ "With two flags",
      TRUE ~ NA
    )
  )
```

2.3 Data Summaries

First, I provide a short description of the other variables in their data. Note that the size ordering of types of ships discussed below is:

$$\text{Handysize} < \text{Supramax} < \text{Ultramax} < \text{Panamax} < \text{Capesize}$$

- `DWT`: The deadweight tonnage, which is the maximum weight each Supramax and Ultramax ship can safely carry in metric tons. This gives a measure of how much cargo, fuel, fresh water, ballast water, provisions, and crew a ship can handle.
- `Yas`: The age in years. New Supramax and Ultramax ships were recorded as having age 1.
- `Enginges RPM`: Revolutions per minute of each Supramax and Ultramax ship's main propulsion engine. Generally speaking, lower-RPM engines are more expensive because they are more fuel efficient.
- `BACI`: The Baltic Capesize Index. This describes freight rates for Capesize dry bulk ships, which reflects supply and demand. When BACI is high, Capesize ships are more profitable in shipping, which could positively affect Supramax and Ultramax ship values as demand for dry bulk commodities might be high and large shipments could be split into several Supramax and Ultramax shipments.
  Note: This reports the current BACI. The data also contain the BACI form 1, 2, and 3 months before the sale (`BACI-1`, `BACI-2`, `BACI-3`).
- `BPNI`: The Baltic Panamax Index. This describes the cost to ship dry bulk commodities (e.g., coal or grain) on Panamax class ships. When BPNI is high, we expect Panamax ship prices to go up. Supramax and Ultramax ship values tend to follow this trend as charterers switch to these smaller vessels when Panamax rates become too expensive.
  Note: This reports the current BPNI. The data also contain the BPNI form 1, 2, and 3 months before the sale (`BPNI-1`, `BPNI-2`, `BPNI-3`).
- `BSIS`: The Baltic Supramax Index. This describes freight rates for Supramax vessels, which reflects supply and demand. When BSIS is high, Supramax and Ultramax prices are usually higher because they are more profitable in shipping.
  Note: This reports the current BSIS. The data also contain the BSIS form 1, 2, and 3 months before the sale (`BSIS-1`, `BSIS-2`, `BSIS-3`).
- `BHSI`: The Baltic Handysize Index. This describes freight rates for handysize vessels. When BHSI is high, Supramax and Ultramax ship prices are usually higher because the prices move together.
  Note: This reports the current BHSI. The data also contain the BHSI form 1, 2, and 3 months before the sale (`BHSI-1`, `BHSI-2`, `BHSI-3`).
- `LIBOR`: The London InterBank Offered Rate. This describes the average reported interest rate at which banks would lend US dollars for 3 months.

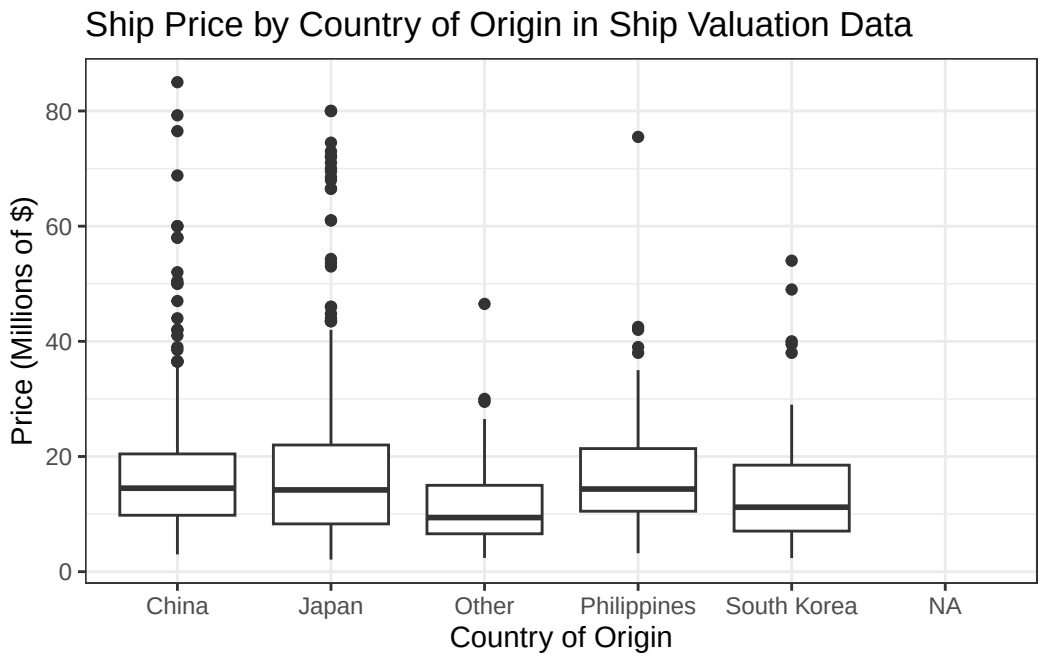
2.3.1 Part A

Choose a categorical variable. Create a plot and corresponding numerical summary to assess whether or how ship prices vary across the levels of the selected categorical variable.

Solution: For the visualization of any categorical variable, I choose to analyze the variable `OriginCountry`. I use a box plot to compare price across categories.

```
ggplot(dat.bal.cleaned) +                                  # Specify Data
  geom_boxplot(aes(x = OriginCountry,                          # Specify x-axis
               y = price)) +                               # Specify y-axis
  theme_bw() +                                             # A cleaner theme
  labs(x = "Country of Origin",                            # Specify axis labels
```

```
       y = "Price (Millions of $)") +
  ggtitle("Ship Price by Country of Origin in Ship Valuation Data") # Plot title
```



I then produce a corresponding numerical summary:

```
bal.summary = dat.bal.cleaned |>
  group_by(OriginCountry) |>
  summarize(
    n = n(),
    `Missing Price` = sum(is.na(price)),
    `Mean Price` = mean(price, na.rm = TRUE),
    `SD Price` = sd(price, na.rm = TRUE),
    `Median Price` = median(price, na.rm = TRUE),
    `IQR Price` = IQR(price, na.rm = TRUE)
  )

knitr::kable(bal.summary)
```

OriginCountry	n	Missing Price	Mean Price	SD Price	Median Price	IQR Price
China	613	0	16.80350	10.391663	14.50	10.650
Japan	898	0	16.86071	11.846689	14.20	13.700
Other	103	0	11.80471	7.375389	9.40	8.417
Philippines	130	0	17.07369	10.255985	14.35	10.875
South Korea	75	0	14.63958	10.316007	11.20	11.450
NA	200	200	NaN	NA	NA	NA

From the numerical summary, we see that the Phillipines has the highest mean price (around 17 million dollars), followed by Japan and China (16.86 and 16.8 million dollars respectively). Countries that fall under other have a noticeably lower average price of around 11.8 million dollars. The median prices are more similar than the means, though "other" countries and South Korea have very low medians (9.4 and 11.2 million dollars respectively).

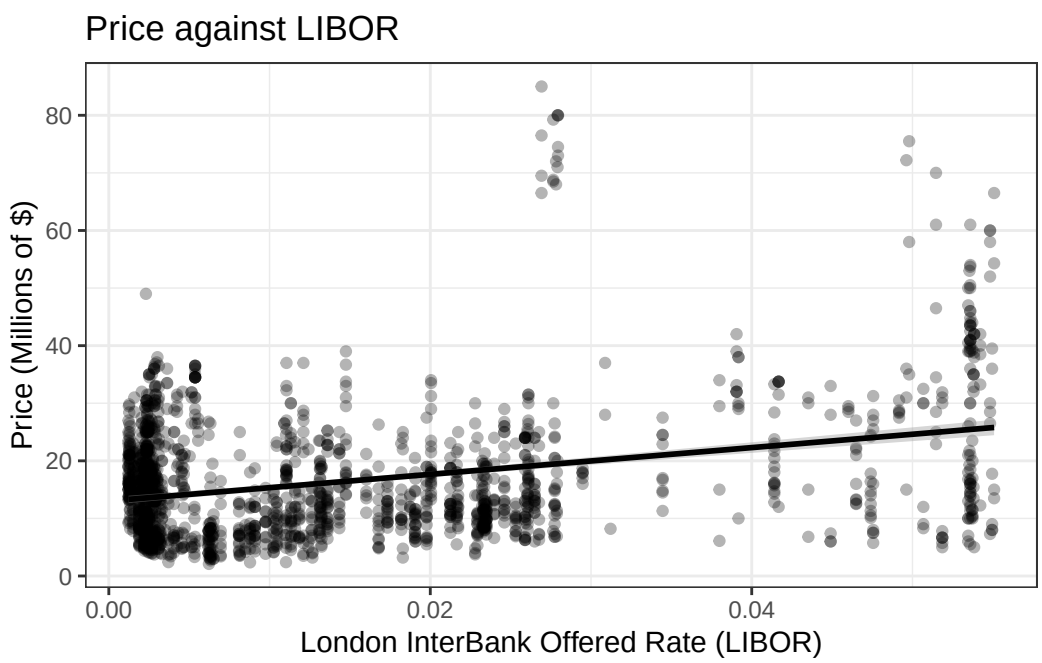
We can also see that Japan has the highest variability in terms of standard deviation and IQR, with "other" countries showing the least variability.

2.3.2 Part B

Choose a quantitative variable. Create a plot and corresponding numerical summary to assess whether or how ship prices vary with the selected quantitative variable.

Solution: For the visualization of any quantitative/continuous variable, I choose to analyze the variable `LIBOR`, which reports the average interest rate at which banks would lend US dollars for 3 months. I use a scatterplot to visualize the relationship between this and price.

```
ggplot(dat.bal.cleaned) +
  geom_point(aes(x = `LIBOR`,
                 y =  price), alpha = 0.3) +
  geom_smooth(aes(x = `LIBOR`,
                  y = price),
              method = "lm",
              color = "black") +
  theme_bw() +
  labs(x = "London InterBank Offered Rate (LIBOR)",
       y = "Price (Millions of $)") +
  ggtitle("Price against LIBOR")
```



Note that an alpha of 0.3 is specified to add transparency to our points. This helps tackle the problem of densely concentrated data (particulary close to 0.00).

To create the numerical summary, we use the code below:

Table 2: Ship Price vs LIBOR in Ship Valuation Data

n	Missing LIBOR	Missing Price	Correlation	Mean LIBOR	SD LIBOR	Mean Price	SD Price
2019	200	200	0.323471	0.0149213	0.015402	16.47878	11.04333

We can see that there is a weak, positive, linear correlation between LIBOR and ship price (corr = 0.323471). The scatterplot shows that most observations occur with LIBOR values close to 0 (`mean LIBOR` = .0149213). The average price of ships from this dataset is around 16.5 million dollars, with a significant amount of spread (SD = 11.04333).

References

Bal, Elif Tuçe, Ercan Akan, and Huseyin Gencer. 2025. "A Novel Approach to Ship Valuation Prediction: An Application to the Supramax and Ultramax Secondhand Markets." *PLoS One* 20 (5): e0319073.

Gupta, Mohit, and Mahmoud Shorman. 2025. "Cytomegalovirus Infections." In *StatPearls [Internet]*. StatPearls Publishing.

Kitsberg, Yaarit, Aharon Nachshon, Tamar Arazi, et al. 2025. "Viral Entry Shapes HCMV Latency Establishment." *Nature Communications*.

Wickham, Hadley, Mara Averick, Jennifer Bryan, et al. 2019. "Welcome to the tidyverse." *Journal of Open Source Software* 4 (43): 1686. https://doi.org/10.21105/joss.01686.

Wickham, Hadley, and Jennifer Bryan. 2025. *Readxl: Read Excel Files*. https://doi.org/10.32614/CRAN.package.readxl.